Engineering the Agam Compiler & Language Programming Guide

A Complete Textbook, Architecture Reference & Language User Guide

---

Engineering the Agam Compiler & Language Programming Guide

A Comprehensive Textbook, Architecture Reference & Language User Guide

---

■ Book Overview

This textbook provides a comprehensive, structured guide to building modern optimizing compilers and programming in **Agam**. Using the production **Agam Compiler** (`crates/{core,middle,backends,runtime,tooling}`) as its primary implementation model, this volume bridges foundational literature with industrial software engineering practice.

Interactive Online & Offline HTML Book

Build or serve this entire documentation suite as an interactive, searchable web book using **mdBook**:

```
cd doc
mdbook serve --open
```

Core Literature Integration

Every chapter integrates theoretical principles and practical patterns from seven landmark texts in systems programming and compiler design:

- **The C Programming Language (K&R)** by Brian W. Kernighan & Dennis M. Ritchie

- **Crafting Interpreters** by Robert Nystrom

- **Language Implementation Patterns** by Terence Parr

- **Engineering a Compiler** by Keith D. Cooper & Linda Torczon

- **Modern Compiler Implementation in C (Tiger Book)** by Andrew W. Appel

- **LLVM Code Generation: A Deep Dive** by Quentin Colombet

- **LLVM Techniques, Tips, and Best Practices** by Kai Nacke & Amy Kwan

---

■ Complete Table of Contents

Front Matter

- **Preface & Reader Roadmap**: Book objectives, literature mapping, and pedagogical tracks.

- **One-Page Syntax Cheat Sheet**: Quick reference card for Agam syntax, tensors, effects, and CLI tools.

Part I: Systems Programming & Low-Level Foundations

Grounded in Kernighan & Ritchie (K&R C)

- **Chapter 1: The C Execution & Memory Model**: Stack vs. Heap layout, pointer arithmetic, struct alignment, and padding calculations.

- **Chapter 2: Hardware Architecture, Calling Conventions & System ABIs**: System V AMD64 vs Windows x64 ABIs, register usage, stack frame construction, and runtime C ABI bindings.

Part II: Language Design & Frontend Mechanics

Grounded in Nystrom (Crafting Interpreters) & Parr (Language Implementation Patterns)

- **Chapter 3: Lexical Analysis & Token Scanning**: Token streams, UTF-8 scanning, and source position tracking (`Span`, `SourceId`).

- **Chapter 4: Parsing Theory & Pratt Parsing Mechanics**: Top-down operator precedence parsing, binding powers, and statement parsing.

- **Chapter 5: Abstract Syntax Trees & Grammar Representation**: Recursive AST design, expression variants, pattern match syntax, and effect nodes.

- **Chapter 6: Symbol Tables, Lexical Scopes & Type Inference**: Nested scope graphs, symbol resolution, type propagation, and effect checking.

Part III: Compiler Architecture & Optimization Theory

Grounded in Cooper & Torczon (Engineering a Compiler) & Appel (Tiger Book)

- **Chapter 7: High-Level & Medium-Level Intermediate Representations (HIR & MIR)**: AST lowering, desugaring passes, HIR layout, and MIR control flow structure.

- **Chapter 8: Control Flow Graphs & Static Single Assignment (SSA) Form**: Basic Blocks, ϕ-node placement, dominance frontiers, and definition-use chains.

- **Chapter 9: Middle-End Optimization Passes**: Constant folding, dead code elimination (DCE), loop invariant code motion (LICM), and function inlining.

- **Chapter 10: Lowering Functional & Effectful Semantics**: Closure conversion, decision-tree pattern matching, and algebraic effect suspension frames.

Part IV: LLVM Backend & Code Generation Infrastructure

Grounded in Colombet (LLVM Code Generation) & Nacke & Kwan (LLVM Techniques)

- **Chapter 11: Emitting Textual & Bitcode LLVM IR**: Mapping MIR to LLVM IR, context setup, builder patterns, and bitcode emission.

- **Chapter 12: Modern PassManager & In-Process JIT Engines**: Optimization pass pipelines (-O0 to -O3), ORC JIT, and Cranelift execution.

- **Chapter 13: LLVM Backend Architecture: SelectionDAG, GlobalISel & MachineIR**: SelectionDAG vs. GlobalISel pipelines, MachineIR (MIR layer), and TableGen (`.td`) files.

- **Chapter 14: Register Allocation Algorithms & Machine Code (MC) Layer**: Graph coloring vs greedy allocation, spilling, MC layer, and native binary generation.

Part V: The Agam Compiler Architecture & Features

Production System Architecture & Advanced Design

- **Chapter 15: End-to-End Agam Compiler Pipeline Walkthrough**: Full source-to-binary compilation lifecycle and driver coordination (`agam_driver`).

- **Chapter 16: Advanced Language Features: Native Tensors & Algebraic Effects**: Hardware-accelerated tensor primitives and algebraic effect handler implementation.

- **Chapter 17: Incremental Compilation Daemon & Sandboxed Runtime**: `DaemonSession` warm state caching, snapshot invalidation, and OS-level sandboxing (JobObject/prctl).

- **Chapter 18: Indic Grammatical Design Principles (P■■ini & Tolk■ppiyam)**: P■■ini's A■■■dhy■y■ and Tolk■ppiyam rules: Dh■tu root verbs, Vibhakti roles, and Type Sandhi rules.

Part VI: The Agam Language Programming Guide

Complete Application Programming Guide (Basic to Advanced)

- **Chapter 19: Getting Started & Basics of Agam**: Hello World, variables, mutability, primitives, function signatures.

- **Chapter 20: Control Flow, Structs & Collections**: Conditionals, loops, `struct`, methods, arrays, tuples.

- **Chapter 21: Tagged Union Enums, Pattern Matching & Error Handling**: Payload enums, pattern matching (`match`), `Option[T]`, `Result[T, E]`.

- **Chapter 22: First-Class Tensors & Numerical AI Operations**: `Tensor` primitives, matrix multiplication, shape broadcasting, neural net layers.

- **Chapter 23: Algebraic Effect Handlers in Depth**: `effect`, `perform`, `handle`, `resume`, async non-blocking control flow.

- **Chapter 24: Modules, Package Management (`agam.toml`) & FFI**: Package manifests, imports, C & Python FFI bindings.

- **Chapter 25: Metaprogramming, REPL, Notebooks & Tooling**: `agamc repl`, headless agent execution (`agamc exec`), `agamc fmt`, `agamc lint`.

- **Chapter 25b: Real-World Agam Code Cookbook**: Production recipes for Web API with Effects, ML Tensor Pipelines, CLI tools.

Part VII: Advanced Tooling, Testing & Ecosystem Engineering

Production Infrastructure & Tooling Architecture

- **Chapter 26: Diagnostic Engineering, Spans & Error Recovery**: Diagnostic data models (`agam_errors`), span tracking, snippet rendering, error recovery.

- **Chapter 27: Testing Methodologies, Fuzzing & Differential Verification**: Multi-tier testing, test harnesses (`agam_test`), JIT vs LLVM differential testing.

- **Chapter 28: Language Server Protocol (LSP) Architecture**: JSON-RPC server (`agam_lsp`), publishDiagnostics, hover tooltips, go-to-definition, autocomplete.

- **Chapter 29: Source Code Formatting Engine Architecture (`agam_fmt`)**: Formatter pipeline (`agam_fmt`), CST traversal, indentation rules, line breaking.

- **Chapter 30: Cross-Compilation, Target Triplets & Target Packs**: Target triplets, Android ARM64 target packs, cross-linking staging (`agam_pkg`).

- **Chapter 31: Compiler Profiling & Performance Measurement**: Profiling harnesses (`agam_profile`), throughput measurement, flamegraph phase timings.

Back Matter

- **Appendix A: Comprehensive Agam Crate Map**: Physical crate boundaries, dependencies, and API surfaces.

- **Appendix B: Annotated Bibliography & Reading List**: Detailed references and study guides.

- **Appendix C: Glossary of Compiler & Indic Design Terms**: Glossary of compiler engineering and Indic grammatical terminology.

Front Matter: Preface & Pedagogical Roadmap

Title Page

Engineering the Agam Compiler

From Systems Foundations to Advanced LLVM Infrastructure

---

1. Preface

Compilers are often viewed as mystifying software systems reserved for specialist theoretical computer scientists. However, modern industrial compilers are disciplined engineering pipelines built upon structured transformations, graph algorithms, and formal execution contracts.

The purpose of this textbook is to provide a complete, accessible, yet rigorous guide to compiler engineering. By pairing classic foundational literature with the concrete implementation of the **Agam Compiler** (`crates/{core,middle,backends,runtime,tooling}`), readers learn not only *why* compiler algorithms work theoretically, but *how* they are implemented in production Rust code.

---

2. Theoretical Framework & Classic Literature

This book integrates concepts across seven landmark compiler engineering works:

1. **The C Programming Language (K&R)**: Teaches the low-level machine execution model, pointer arithmetic, memory alignment, and standard C ABI calling conventions.

2. **Crafting Interpreters (Robert Nystrom)**: Demonstrates modern, readable frontend implementation including lexing, Pratt parsing, and object mechanics.

3. **Language Implementation Patterns (Terence Parr)**: Provides structural design patterns for AST trees, symbol tables, nested lexical scopes, and type checking.

4. **Engineering a Compiler (Keith D. Cooper & Linda Torczon)**: Explores modern intermediate representations (IR), Control Flow Graphs (CFG), SSA form, register allocation, and instruction scheduling.

5. **Modern Compiler Implementation in C (Andrew W. Appel)**: Establishes pipelines for translating high-level functional concepts into imperative IRs and target assembly.

6. **LLVM Code Generation: A Deep Dive (Quentin Colombet)**: Details LLVM code generation infrastructure, MachineIR (MIR), SelectionDAG/GlobalISel, TableGen files, and backend target generation.

7. **LLVM Techniques, Tips, and Best Practices (Kai Nacke & Amy Kwan)**: Demonstrates practical C++ LLVM API usage, AST-to-LLVM-IR translation, PassManager configuration, and JIT compilation.

---

3. Pedagogical Roadmaps

Depending on your prior experience, follow these recommended reading tracks:

Track 1: Beginner (Systems & Frontend Foundations)

- **Part I**: Chapters 1–2 (C memory model, calling conventions, stack frames)

- **Part II**: Chapters 3–6 (Lexing, Pratt parsing, AST, symbol resolution, type checking)

Track 2: Intermediate (Compiler Middle-End & Optimization)

- **Part II**: Chapters 5–6 (AST nodes & semantic checking)

- **Part III**: Chapters 7–10 (HIR/MIR, Control Flow Graphs, SSA form, middle-end optimizations)

Track 3: Advanced (LLVM Backend Engineering & Compiler Architecture)

- **Part III**: Chapters 8–10 (SSA transformations & functional lowerings)

- **Part IV**: Chapters 11–14 (LLVM IR emission, PassManager, GlobalISel, register allocation)

- **Part V**: Chapters 15–18 (Agam compiler architecture, daemon compilation, sandboxing, Indic design principles)

Agam Language Syntax Cheat Sheet

A One-Page Syntax & CLI Quick Reference for Agam Developers

---

1. Variables & Primitive Types

```
let x: Int = 42;                // Immutable integer
let mut count = 0;              // Mutable integer (type inferred)
let ratio: Float = 3.14;        // 64-bit float
let active: Bool = true;        // Boolean
let name: String = "Agam";      // UTF-8 String
const MAX_LIMIT: Int = 1000;    // Compile-time constant
```

---

2. Functions & Control Flow

```
// Standard function
fn add(a: Int, b: Int) -> Int {
    return a + b;
}

// Implicit expression return syntax
fn square(n: Int) -> Int => n * n;

// Conditionals as expressions
let status = if score >= 50 { "Pass" } else { "Fail" };

// Loops
while count < 10 { count = count + 1; }
for i in 0..5 { println(i.to_string()); }
```

---

3. Structs & Methods

```
struct Point { x: Float, y: Float }

impl Point {
    fn origin() -> Point => Point { x: 0.0, y: 0.0 };
    fn distance(self) -> Float => (self.x * self.x + self.y * self.y).sqrt();
}
```

---

4. Enums & Pattern Matching

```
enum Status {
    Idle,
    Processing(percent: Int),
    Error(String),
}

let msg = match status {
    Status.Idle => "System Idle",
    Status.Processing(p) => "Progress: " + p.to_string() + "%",
    Status.Error(err) => "Error: " + err,
};
```

---

5. First-Class Tensors

```
let A: Tensor[Float, 2x2] = Tensor.from_array([[1.0, 2.0], [3.0, 4.0]]);
let B = Tensor.ones([2, 2]);

let C = A * B;             // Matrix multiplication
let D = Tensor.relu(C);    // Activation function
```

---

6. Algebraic Effects

```
effect Logger { fn log(msg: String) -> Nil; }

fn compute() {
    perform Logger.log("Computing...");
}

fn main() {
    handle compute() {
        Logger.log(msg) => { println("LOG: " + msg); resume(); }
    }
}
```

---

7. CLI Reference (`agamc`)

```
agamc build main.agam                # Build native binary
agamc run main.agam                  # Compile and execute
```

```
agamc check main.agam                 # Fast type check
agamc repl                            # Launch interactive JIT REPL
agamc fmt main.agam                   # Format source code
agamc lint main.agam                  # Run static linter
agamc dev                             # Start daemon incremental loop
agamc exec --json '{"source":"..."}'    # Sandboxed headless execution
```

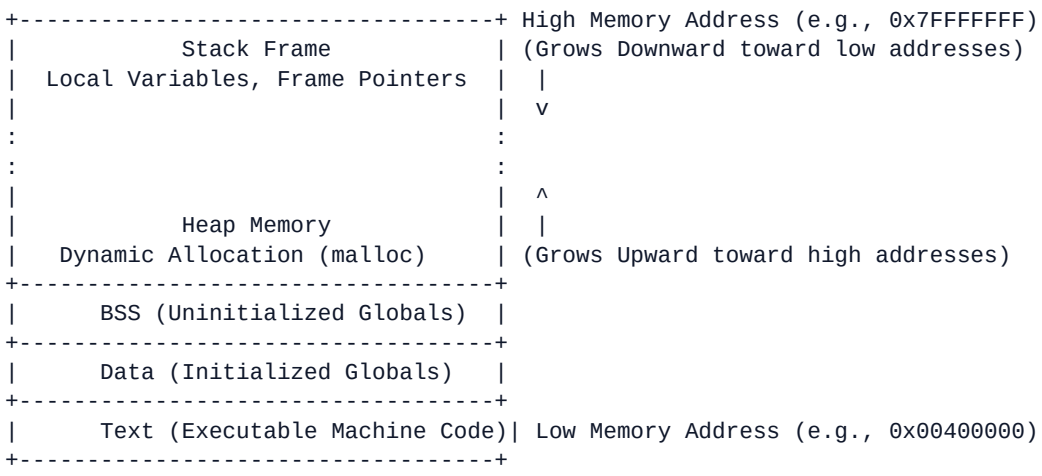
Chapter 1: The C Execution & Memory Model

> **Core Literature Grounding**: *The C Programming Language (K&R)* by Brian W. Kernighan & Dennis M. Ritchie

> **Compiler Module Focus**: `agam_runtime`, `agam_codegen`

1.1 Physical Memory Layout

Compilers translate abstract programming semantics into raw memory operations. Physical process memory allocated by the operating system is partitioned into several distinct segments:



- **Text Segment**: Contains immutable binary instructions executed directly by the CPU instruction pointer (`rip`).

- **Data Segment**: Holds initialized global and static variables.

- **BSS Segment**: Holds uninitialized global variables, zeroed by the OS kernel upon process launch.

- **Heap**: Dynamic memory managed programmatically via allocators (`malloc`/`free`, bump allocators).

- **Stack**: Automatic memory managed via CPU stack pointer manipulation (`rsp`).

1.2 Data Alignment & Struct Padding

Modern CPU architectures access multi-byte primitive types (e.g., 32-bit integers, 64-bit pointers) most efficiently when located at addresses divisible by their size. Unaligned memory accesses can trigger

performance penalties or CPU bus faults.

Struct Alignment Rule

The compiler calculates struct layout offsets using alignment formulas:

$$\text{Offset}(X_{i+1}) = \text{AlignUp}(\text{Offset}(X_i) + \text{sizeof}(X_i), \text{AlignOf}(X_{i+1}))$$

Layout Example

Consider a composite type definition:

```
struct SystemHeader {
    char  id;        // 1 byte  (Offset 0)
                     // 3 bytes padding inserted by compiler
    int   flags;     // 4 bytes (Offset 4)
    short version;   // 2 bytes (Offset 8)
                     // 2 bytes padding inserted to align total size to 4-byte boundary
};                   // Total Size: 12 bytes
```

In `agam_sema` and `agam_mir`, struct layout calculators compute these exact padding byte offsets to guarantee ABI compatibility with target C runtimes.

1.3 Pointer Arithmetic & Memory Addressing

Pointers represent physical memory addresses. In C and generated target IR, adding an integer `k` to a pointer `p` scales `k` by the size of the referenced type T:

$$\text{Address}(p + k) = \text{Address}(p) + k \times \text{sizeof}(T)$$

Compilers emit pointer offset calculations explicitly using indexed memory operand instructions (e.g., `mov rax, [rbx + rdi*8]`).

Chapter 2: Hardware Architecture, Calling Conventions & System ABIs

> **Core Literature Grounding**: *The C Programming Language (K&R)* by Brian W. Kernighan & Dennis M. Ritchie

> **Compiler Module Focus**: `agam_runtime`, `agam_codegen`

2.1 The Application Binary Interface (ABI)

An **Application Binary Interface (ABI)** establishes the machine-level contract for function invocation, parameter passing, return value handling, register preservation, and stack alignment across compiled modules.

Without a standardized ABI, compiled binary code generated by one toolchain (e.g., `agamc`) could not invoke native host system APIs (e.g., C runtime, Win32, POSIX libc).

2.2 Standard Target ABIs

1. System V AMD64 ABI (Linux, macOS, BSD, Android)

- **Register Assignment**: The first 6 integer or pointer parameters are passed in CPU registers:

1. `rdi`
2. `rsi`
3. `rdx`
4. `rcx`
5. `r8`
6. `r9`

- **SIMD/Floating-Point**: Passed in `xmm0` through `xmm7`.

- **Overflow Arguments**: Parameters beyond 6 are pushed onto the stack in right-to-left order.

- **Return Value**: Placed in `rax` (integer/pointer) or `xmm0` (floating point).

- **Stack Alignment**: The stack pointer `rsp` must be 16-byte aligned before executing a `call` instruction.

2. Windows x64 ABI (Microsoft Windows)

- **Register Assignment**: The first 4 integer or pointer parameters are passed in:

1. `rcx`

2. `rdx`

3. `r8`

4. `r9`

- **Shadow Space**: The caller **must** allocate 32 bytes of "shadow space" (home space) on the stack immediately before calling a function, giving the callee space to spill register arguments if needed.

2.3 Stack Frame Mechanics

When a function executes, it builds a stack frame managed by the Stack Pointer (`rsp`) and Base/Frame Pointer (`rbp`):

```
+----------------------+ High Memory Addresses
| Parameter N          |
| ...                  |
| Parameter 7          |
+----------------------+
| Return Address       | <- Automatically pushed by hardware `call` instruction
+----------------------+
| Saved Frame Ptr (rbp) | <- Pushed by Function Prologue (`push rbp`)
+----------------------+ <- Base Pointer `rbp` points here
| Local Variable 1     |
| Local Variable 2     |
| Temp Storage         | <- Space reserved by prologue (`sub rsp, FrameSize`)
+----------------------+ High-water mark Stack Pointer `rsp` points here
```

Function Prologue & Epilogue

```
; Function Prologue
push rbp            ; Save caller frame pointer
mov  rbp, rsp       ; Establish new frame pointer
sub  rsp, 32        ; Allocate 32 bytes for local variables

; ... Function Body ...

; Function Epilogue
mov  rsp, rbp       ; Deallocate local stack frame
pop  rbp            ; Restore caller frame pointer
ret                 ; Return control to caller
```

2.4 FFI & Runtime Interop in Agam

Agam's runtime system (`agam_runtime`) exposes a C-compatible ABI interface (`#[no_mangle] pub extern "C"` functions) allowing native binaries, C/C++ libraries, and external Python FFI adapters (`agam_ffi`) to invoke Agam runtime capabilities seamlessly.

Chapter 3: Lexical Analysis & Token Scanning

> **Core Literature Grounding**: *Crafting Interpreters* (Chapter 4) by Robert Nystrom

> **Compiler Module Focus**: `agam_lexer`, `agam_errors`

3.1 Role of the Lexer

The **Lexer** (or Scanner) forms the first stage of the compiler frontend. It converts an unformatted stream of UTF-8 source text into a sequential stream of structured **Tokens**, stripping whitespace and comments while preserving positional metadata.

3.2 Token Structure & Source Span Attribution

To generate diagnostic error reports with source code underlining, every token must record its physical location in the input source file.

In `agam_lexer`, tokens are paired with a `Span`:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct SourceId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Span {
    pub start: u32,       // Byte offset of first character
    pub end: u32,         // Byte offset past last character
    pub source_id: SourceId, // Unique ID of input source file
}

#[derive(Debug, Clone, PartialEq)]
pub struct Token {
    pub kind: TokenKind,
```

```
    pub span: Span,
}
```

3.3 Lexer Implementation Techniques

State Machine Character Scanning

The scanner iterates through characters using a single lookahead pointer:

```
pub enum TokenKind {
    // Keywords
    Fn, Let, Perform, Handle, Effect, Match,
    // Literals
    Identifier(String), Integer(i64), Float(f64), StringLit(String),
    // Operators & Punctuation
    Plus, Minus, Star, Slash, Equal, EqualEqual, Colon, Arrow,
    // Control
    EOF, Error(String),
}
```

When scanning multi-character operators (e.g., = vs. ==, ->), the scanner inspects the lookahead character (`peek()`) to determine whether to advance:

```
match current_char {
    '=' => {
        if self.peek_char() == '=' {
            self.advance();
            TokenKind::EqualEqual
        } else {
            TokenKind::Equal
        }
    }
    '-' => {
        if self.peek_char() == '>' {
            self.advance();
            TokenKind::Arrow
        } else {
            TokenKind::Minus
        }
    }
    _ => ...
}
```

3.4 Resilient Diagnostic Error Recovery

If the lexer encounters invalid UTF-8 sequences or unrecognized characters, it does not abort immediately. Instead, it emits an `Error` token variant alongside an `agam_errors` diagnostic, allowing the scanner to continue processing subsequent tokens so the compiler can report multiple errors in a single build pass.

Chapter 4: Parsing Theory & Pratt Parsing Mechanics

> **Core Literature Grounding**: *Crafting Interpreters* (Chapter 17) by Robert Nystrom

> **Compiler Module Focus**: `agam_parser`, `agam_ast`

4.1 Parsing Paradigms

A **Parser** converts a linear stream of tokens into a tree structure representing grammatical hierarchy: the **Abstract Syntax Tree (AST)**.

Traditional parsing strategies include:

- **LL(k) Recursive Descent**: Simple for statements, but struggles with operator precedence without producing deep, inefficient call stacks.

- **LR/LALR Parsers (Yacc/Bison)**: Table-driven parsers generated by external tools, often producing difficult-to-debug error messages.

- **Pratt Parsing (Top-Down Operator Precedence)**: Combines recursive descent simplicity with elegant, flat operator precedence resolution.

Agam uses **Pratt Parsing** for all expressions in `agam_parser`.

4.2 Pratt Parsing Architecture

Pratt parsing associates parsing functions with individual token types based on their position in an expression:

Null Denotation (Nud / Prefix Parselet)

Invoked when a token appears at the **beginning** of an expression:

- Literals: `42`, `"hello"`

- Identifiers: `x`, `total`

- Prefix operators: `-x`, `!flag`

- Effect invocations: `perform Logger.log("message")`

- Grouping: `(a + b)`

Left Denotation (Led / Infix Parselet)

Invoked when a token appears **between** two expressions:

- Infix binary operators: `a + b`, `x * y`, `x == y`

- Postfix operators: `x++`

- Function calls: `f(arg1, arg2)`

- Field accesses: `object.field`

4.3 Binding Power & Precedence Resolution

Each infix operator is assigned a **Left Binding Power (LBP)** and a **Right Binding Power (RBP)** integer:

| Operator | Left Binding Power (LBP) | Right Binding Power (RBP) | Associativity |

| :--- | :--- | :--- | :--- |

| +, - | 10 | 11 | Left-associative |

| *, / | 20 | 21 | Left-associative |

| ^ (power) | 31 | 30 | Right-associative |

| ==, != | 5 | 6 | Non-associative / Left |

Pratt Algorithm Core Loop

```
pub fn parse_expr(&mut self, current_bp: u8) -> Result<Expr, ParseError> {
    let token = self.advance();

    // 1. Execute Prefix Parselet (Nud)
    let mut left = match token.kind {
        TokenKind::Integer(val) => Expr::Literal(Literal::Int(val)),
        TokenKind::Minus => {
            let rhs = self.parse_expr(BindingPower::Prefix)?;
            Expr::Unary { op: UnOp::Neg, expr: Box::new(rhs) }
        }
        TokenKind::Perform => self.parse_perform_expr()?,
        _ => return Err(ParseError::UnexpectedToken(token)),
    };

    // 2. Loop while next token's infix binding power exceeds current_bp
    while let Some(next_token) = self.peek() {
        let (left_bp, right_bp) = self.infix_binding_power(&next_token.kind);
        if left_bp <= current_bp {
            break;
        }

        self.advance(); // Consume operator token
```

```
            // 3. Execute Infix Parselet (Led)
            left = match next_token.kind {
                TokenKind::Plus => {
                    let rhs = self.parse_expr(right_bp)?;
                    Expr::Binary { op: BinOp::Add, lhs: Box::new(left), rhs: Box::new(rhs) }
                }
                TokenKind::Star => {
                    let rhs = self.parse_expr(right_bp)?;
                    Expr::Binary { op: BinOp::Mul, lhs: Box::new(left), rhs: Box::new(rhs) }
                }
                _ => break,
            };
        }

        Ok(left)
    }
```

4.4 Statement & Module Parsing

Statement parsing uses standard recursive descent, building block lists for functions, variable declarations (`let`), pattern matches (`match`), and effect handler expressions (`handle`).

Chapter 5: Abstract Syntax Trees & Grammar Representation

> **Core Literature Grounding**: *Language Implementation Patterns* (Chapter 3) by Terence Parr

> **Compiler Module Focus**: `agam_ast`

5.1 Concrete vs. Abstract Syntax Trees

- **Concrete Syntax Tree (CST / Parse Tree)**: Preserves every single token from input source text, including grouping parentheses, commas, semicolons, and comments. CSTs are essential for tools like formatters (`agam_fmt`) and language servers (`agam_lsp`).

- **Abstract Syntax Tree (AST)**: Discards redundant syntactic noise (e.g., matching parentheses, semicolons), preserving only semantic hierarchy. ASTs are optimized for type checking (`agam_sema`) and IR lowering (`agam_hir`).

5.2 AST Node Architecture in Rust (`agam_ast`)

In `agam_ast`, nodes are represented using algebraic data types (`enum` and `struct` definitions):

```
pub struct Module {
    pub name: String,
    pub items: Vec<Item>,
    pub span: Span,
}

pub enum Item {
    Fn(Function),
    Struct(StructDecl),
    Enum(EnumDecl),
    Effect(EffectDecl),
}

pub struct Function {
    pub name: Ident,
    pub params: Vec<Param>,
```

```
    pub return_type: Option<TypeAnnotation>,
    pub body: Block,
    pub span: Span,
}

pub enum Stmt {
    Let { name: Ident, ty: Option<TypeAnnotation>, init: Expr, span: Span },
    Expr(Expr),
    Return(Option<Expr>, Span),
}

pub enum Expr {
    Literal(Literal),
    Ident(Ident),
    Binary { op: BinOp, lhs: Box<Expr>, rhs: Box<Expr>, span: Span },
    Call { callee: Box<Expr>, args: Vec<Expr>, span: Span },
    Perform { effect_name: Ident, payload: Box<Expr>, span: Span },
    Handle { body: Box<Expr>, handlers: Vec<HandlerClause>, span: Span },
    Match { target: Box<Expr>, arms: Vec<MatchArm>, span: Span },
}
```

5.3 AST Visitor & Transformer Patterns

Following Terence Parr's *Language Implementation Patterns*, tree manipulation operations (semantic checking, AST rewrites, diagnostic validation) are decoupled from AST definitions using **Visitor** or **Folder** traits:

```
pub trait AstVisitor {
    fn visit_expr(&mut self, expr: &Expr) {
        walk_expr(self, expr);
    }
    fn visit_stmt(&mut self, stmt: &Stmt) {
        walk_stmt(self, stmt);
    }
}
```

This pattern guarantees clean separation of concerns: syntax trees remain lightweight data structures while passes implement specific compiler operations.

Chapter 6: Symbol Tables, Lexical Scopes & Type Inference Engine

> **Core Literature Grounding**: *Language Implementation Patterns* (Chapters 6–8) by Terence Parr

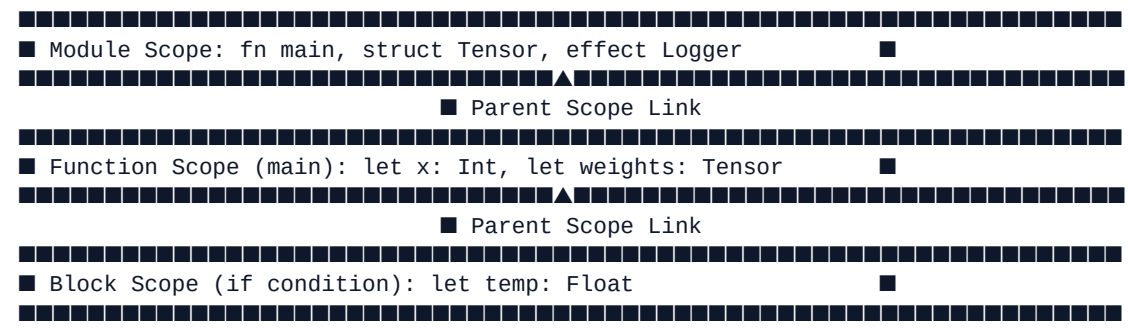
> **Compiler Module Focus**: `agam_sema`

6.1 Symbol Resolution & Lexical Scopes

Before type checking can evaluate expressions, the compiler must resolve every variable, function, or type identifier to its corresponding canonical definition.

Scope Graph Architecture

A **Symbol Table** tracks symbol declarations across nested lexical scope blocks:




```
pub struct SymbolTable {
    scopes: Vec<Scope>,
    current_scope: ScopeId,
}

pub struct Scope {
    parent: Option<ScopeId>,
    symbols: HashMap<String, SymbolInfo>,
}

pub struct SymbolInfo {
    pub name: String,
    pub kind: SymbolKind,
    pub ty: Type,
    pub span: Span,
}
```

6.2 Bidirectional Type Checking & Inference Engine

`agam_sema` enforces static type safety using a bidirectional type checking algorithm:

1. **Type Checking (Top-Down / Synthesize)**: Given an expected target type T, verify that expression E evaluates to T.

2. **Type Inference (Bottom-Up / Analyze)**: Given an expression E without explicit annotations, infer its primitive or composite type T.

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash e_1 + e_2 : \text{Int}}$$

```
pub fn check_expr(&mut self, expr: &Expr, expected: Option<&Type>) -> Result<Type, TypeError> {
    match expr {
        Expr::Literal(Literal::Int(_)) => Ok(Type::Int),
        Expr::Binary { op, lhs, rhs, .. } => {
            let lhs_ty = self.check_expr(lhs, None)?;
            let rhs_ty = self.check_expr(rhs, None)?;

            if lhs_ty != rhs_ty {
                return Err(TypeError::Mismatch { expected: lhs_ty, found: rhs_ty });
            }
            Ok(lhs_ty)
        }
        Expr::Ident(ident) => {
            let symbol = self.symbol_table.lookup(&ident.name)
                .ok_or(TypeError::UndeclaredIdentifier(ident.name.clone()))?;
            Ok(symbol.ty.clone())
        }
        _ => ...
    }
}
```

6.3 Algebraic Effect Checking & Verification

Agam verifies side-effects statically during semantic analysis. If a function contains a `perform EffectName(...)` expression, `agam_sema` verifies that:

1. The effect is declared in the function's signature (`fn run() -> Int ! Logger`), OR

2. The `perform` expression occurs inside an enclosing `handle` block that intercepts `Logger`.

Uncaught or undeclared effects raise compile-time semantic errors (`UnhandledEffect`).

Chapter 7: High-Level & Medium-Level Intermediate Representations (HIR & MIR)

> **Core Literature Grounding**: *Engineering a Compiler* (Chapter 5) by Keith D. Cooper & Linda Torczon

> **Compiler Module Focus**: `agam_hir`, `agam_mir`

7.1 Multi-Stage Intermediate Representations

Compilers decouple language syntax from target machine optimization by introducing intermediate representations. Cooper & Torczon emphasize using intermediate forms tailored to specific compilation passes:

```
AST (Abstract Syntax Tree)
          ■
          ▼ AST Lowering
HIR (High-Level IR - `agam_hir`)
  - Preserves user types, pattern matching, algebraic effects
  - Desugars complex syntactic sugar
          ■
          ▼ Desugaring & Control-Flow Lowering
MIR (Medium-Level IR - `agam_mir`)
  - Control Flow Graph (CFG) of Basic Blocks
  - Explicit temporaries (_1, _2, _3)
  - Static Single Assignment (SSA) form
```

7.2 High-Level IR (HIR - `agam_hir`)

`agam_hir` simplifies complex surface syntax while maintaining high-level type annotations and algebraic effect structures.

Key HIR Responsibilities:

- **Desugaring Compound Control Flow**: Translating `for` loops into `while` loops or basic blocks.

- **Pattern Match Simplification**: Transforming complex nested `match` expressions into explicit decision trees.

- **Explicit Type Resolution**: Replacing inferred types with fully qualified type IDs.

7.3 Medium-Level IR (MIR - `agam_mir`)

`agam_mir` represents code as a control flow graph of basic blocks with explicit temporaries and SSA assignments.

```
pub struct MirFunction {
    pub name: String,
    pub params: Vec<LocalId>,
    pub return_ty: Type,
    pub basic_blocks: IndexVec<BasicBlockId, BasicBlock>,
    pub local_decls: IndexVec<LocalId, LocalDecl>,
}

pub struct BasicBlock {
    pub statements: Vec<Statement>,
    pub terminator: Terminator,
}

pub enum Statement {
    Assign(Place, Rvalue),
    StorageLive(LocalId),
    StorageDead(LocalId),
}

pub enum Terminator {
    Goto(BasicBlockId),
    Branch { cond: Operand, then_block: BasicBlockId, else_block: BasicBlockId },
    SwitchInt { discr: Operand, targets: SwitchTargets },
    Return(Operand),
    YieldEffect { effect_id: u32, payload: Operand, resume_bb: BasicBlockId },
}
```

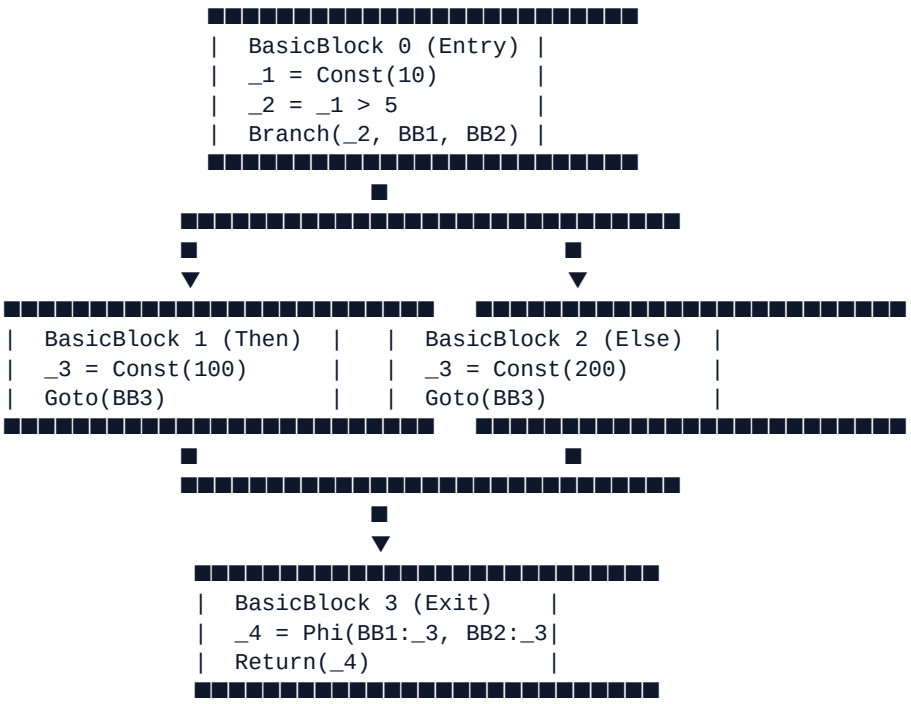
Chapter 8: Control Flow Graphs & Static Single Assignment (SSA) Form

> **Core Literature Grounding**: *Engineering a Compiler* (Chapter 9) by Keith D. Cooper & Linda Torczon

> **Compiler Module Focus**: `agam_mir`

8.1 Control Flow Graph (CFG) Construction

A **Control Flow Graph (CFG)** is a directed graph $G = (V, E)$ where vertices V represent Basic Blocks and edges E represent control flow jumps (`Goto`, `Branch`, `SwitchInt`).



8.2 The SSA Property & ϕ-Nodes

In **Static Single Assignment (SSA)** form:

1. Every temporary variable is defined exactly once.

2. Every use of a variable is dominated by its definition point.

The ϕ-Node (Phi Function)

When control flow branches merge at a join point, values defined in separate predecessor blocks are reconciled using a ϕ-node:

$$\text{\_4} = \phi(\text{BB1: \_3}, \text{BB2: \_3})$$

8.3 Dominance & Dominance Frontiers

Computing minimal SSA form requires dominance analysis over the CFG graph:

1. Dominance Definition

A basic block D dominates block B ($D \text{ dom } B$) if every path from the entry block BB_0 to B must pass through D.

2. Dominance Frontier (DF)

The Dominance Frontier of a block X is the set of all nodes Y such that X dominates a predecessor of Y, but does not strictly dominate Y itself:

$$DF(X) = \{ Y \mid \exists P \in \text{Pred}(Y) \text{ s.t. } X \text{ dom } P \text{ and } X \text{ does not strictly dom } Y \}$$

ϕ-nodes are placed at the iterated dominance frontier $DF^+(B)$ for all basic blocks B containing variable assignments.

Chapter 9: Middle-End Optimization Passes

> **Core Literature Grounding**: *Engineering a Compiler* (Chapters 8 & 10) by Keith D. Cooper & Linda Torczon

> **Compiler Module Focus**: `agam_mir::opt`

9.1 The Middle-End Optimization Pipeline

The goal of middle-end optimization passes is to rewrite MIR control flow graphs into faster, smaller, and memory-efficient forms while preserving original program semantics.

```
Unoptimized MIR
       ■
       ▼  Pass 1: Dead Code Elimination (DCE)
       ▼  Pass 2: Constant Folding & Propagation
       ▼  Pass 3: Function Inlining
       ▼  Pass 4: Loop Invariant Code Motion (LICM)
       ▼
Optimized MIR -> Codegen Backend
```

9.2 Key Optimization Passes

1. Constant Folding & Propagation

Replaces compile-time constant expressions with evaluated literal constants and propagates definitions downstream:

$$\text{\_1 = 10 + 20} \implies \text{\_1 = 30}$$

2. Dead Code Elimination (DCE)

Traverses the CFG definition-use chain to eliminate instructions and basic blocks whose results are never read:

```
// Before DCE
_1 = Const(42); // Unused statement
_2 = Const(100);
Return(_2);

// After DCE
```

```
_2 = Const(100);
Return(_2);
```

3. Function Inlining

Replaces function call sites directly with the target function's basic blocks, eliminating stack frame setup overhead and unlocking scalar optimizations across caller-callee boundaries.

4. Loop Invariant Code Motion (LICM)

Identifies statements inside loop blocks whose operand inputs do not change across iterations, hoisting them into the loop pre-header block.

Chapter 10: Lowering Functional & Effectful Semantics

> **Core Literature Grounding**: *Modern Compiler Implementation in C* (Chapters 14–15) by Andrew W. Appel

> **Compiler Module Focus**: `agam_hir`, `agam_mir`

10.1 Functional to Imperative Lowering

Appel's *Modern Compiler Implementation in C* demonstrates how high-level functional concepts (closures, pattern matching, algebraic effects) are lowered into low-level imperative basic blocks.

10.2 Closure Conversion

When anonymous functions capture variables from enclosing lexical scopes, the compiler transforms them into **explicit closures**:

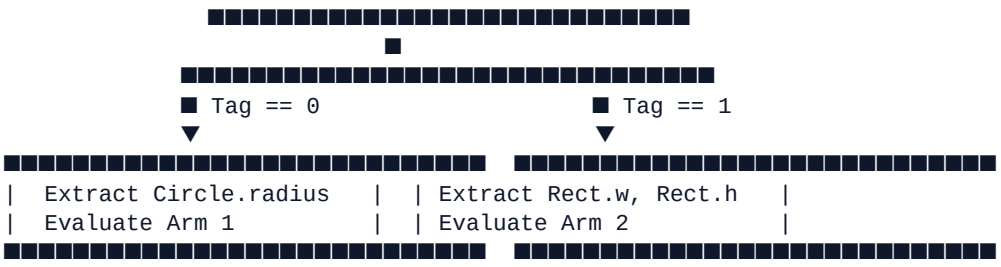
```
High-Level Source:
  let factor = 10;
  let multiplier = fn(x: Int) -> Int { x * factor };

Lowered MIR Transformation:
  struct Closure_1 {
      fn_ptr: fn(*const Closure_1, i64) -> i64,
      env_factor: i64,
  }
```

Captures are explicitly stored inside environment struct payloads, converting indirect function invocations into standard C ABI calls passing the environment pointer.

10.3 Pattern Match Desugaring

Complex pattern matching (`match target { Arm1 => ..., Arm2 => ... }`) is desugared into decision trees composed of `SwitchInt` and `Branch` terminators:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
|  SwitchInt(target.tag)   |
```



10.4 Algebraic Effect Suspension Frames

In Agam, `perform Effect(...)` suspends execution and yields control to an enclosing handler.

During MIR lowering, `perform` operations are converted into `YieldEffect` terminators that:

1. Spill active local temporaries into a stack frame context buffer.

2. Pass the effect payload and resume basic block ID to `agam_runtime_yield`.

3. Allow the handler to resume execution at `resume_bb` upon invocation of `resume()`.

Chapter 11: Emitting Textual & Bitcode LLVM IR

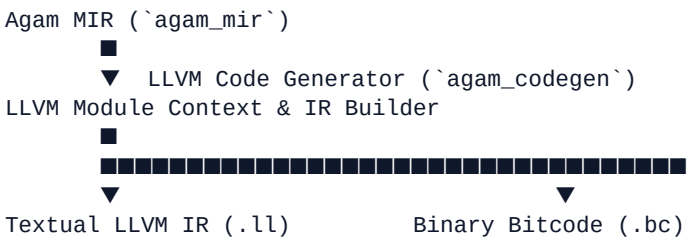
> **Core Literature Grounding**: *LLVM Techniques, Tips, and Best Practices* (Chapters 3–5) by Kai Nacke & Amy Kwan

> **Compiler Module Focus**: `agam_codegen`

11.1 The LLVM IR Code Generation Architecture

`agam_codegen` bridges Agam's Medium-Level IR (MIR) to target-independent **LLVM IR**.

LLVM IR is a strongly typed, RISC-like instruction set in SSA form with infinite virtual registers (`%0`, `%1`, `%2`):



11.2 LLVM Module & Builder Infrastructure

Nacke & Kwan describe the core C++ / Rust LLVM API objects:

- `Context`: Owns core LLVM types, global constants, and thread-local state.

- `Module`: A single translation unit containing functions, global variables, target triple specifications, and data layouts.

- `Builder`: An instruction construction helper that appends newly created LLVM IR instructions onto basic block endpoints.

```
pub struct LLVMEmitter<'ctx> {
    pub context: &'ctx Context,
    pub module: Module<'ctx>,
    pub builder: Builder<'ctx>,
}

impl<'ctx> LLVMEmitter<'ctx> {
    pub fn emit_function(&mut self, mir_fn: &MirFunction) {
```

```
        let ret_ty = self.convert_type(&mir_fn.return_ty);
        let param_tys: Vec<_> = mir_fn.params.iter().map(|p| self.convert_type(&p.ty)).collect();
        let fn_type = ret_ty.fn_type(&param_tys, false);

        let function = self.module.add_function(&mir_fn.name, fn_type, None);
        let entry_bb = self.context.append_basic_block(function, "entry");
        self.builder.position_at_end(entry_bb);

        // Lower MIR basic blocks -> LLVM Basic Blocks
    }
}
```

11.3 Textual IR vs. Bitcode Output

- **Textual LLVM IR (`.ll`)**: Human-readable assembly format used for debugging and inspecting compiler codegen output.

- **Bitcode (`.bc`)**: Compact binary representation passed directly into LLVM optimization passes and linkers.

```
; Textual LLVM IR generated for a simple function
define i64 @calculate_sum(i64 %a, i64 %b) #0 {
entry:
  %0 = add nsw i64 %a, %b
  ret i64 %0
}
```

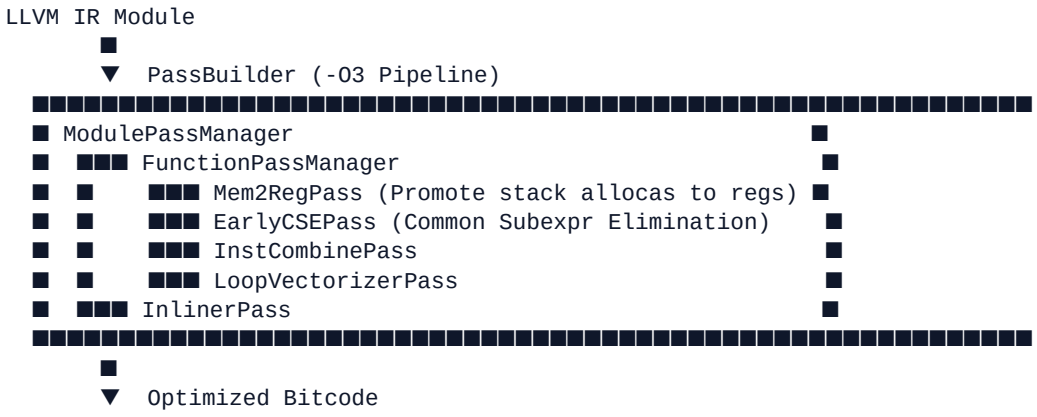
Chapter 12: Modern PassManager & In-Process JIT Engines

> **Core Literature Grounding**: *LLVM Techniques, Tips, and Best Practices* (Chapter 7) by Kai Nacke & Amy Kwan

> **Compiler Module Focus**: `agam_codegen`, `agam_jit`

12.1 Modern LLVM PassManager

LLVM uses the **New PassManager** pipeline to run modular transformations over LLVM IR modules:



Key LLVM Pass Categories:

- **Mem2Reg**: Transforms `alloca` memory locations into LLVM SSA registers (`%1`, `%2`).

- **InstCombine**: Combines redundant instruction sequences into simpler canonical primitives.

- **SLP / Loop Vectorizer**: Emits SIMD instructions (`AVX2`, `AVX-512`, `NEON`) for data-parallel operations.

12.2 In-Process JIT Compilation (`agam_jit`)

For interactive evaluation (`agamc repl`, `agamc exec`), generating `.o` files and invoking host linkers introduces unacceptable latency.

`agam_jit` compiles LLVM IR or MIR directly into executable memory pages (`PROT_READ | PROT_EXEC`) in process memory:

$$\text{LLVM Bitcode / MIR} \xrightarrow{\text{Cranelift / ORC JIT}} \text{Memory Buffer} \xrightarrow{\text{Cast to fn()}} \text{Direct Invocation}$$

```
pub struct AgamJitEngine {
    // Cranelift / LLVM ORC JIT instance
}

impl AgamJitEngine {
    pub unsafe fn execute_function(&mut self, fn_name: &str) -> Result<i64, JitError> {
        let symbol_ptr = self.lookup_symbol(fn_name)?;
        let func: extern "C" fn() -> i64 = std::mem::transmute(symbol_ptr);
        Ok(func())
    }
}
```

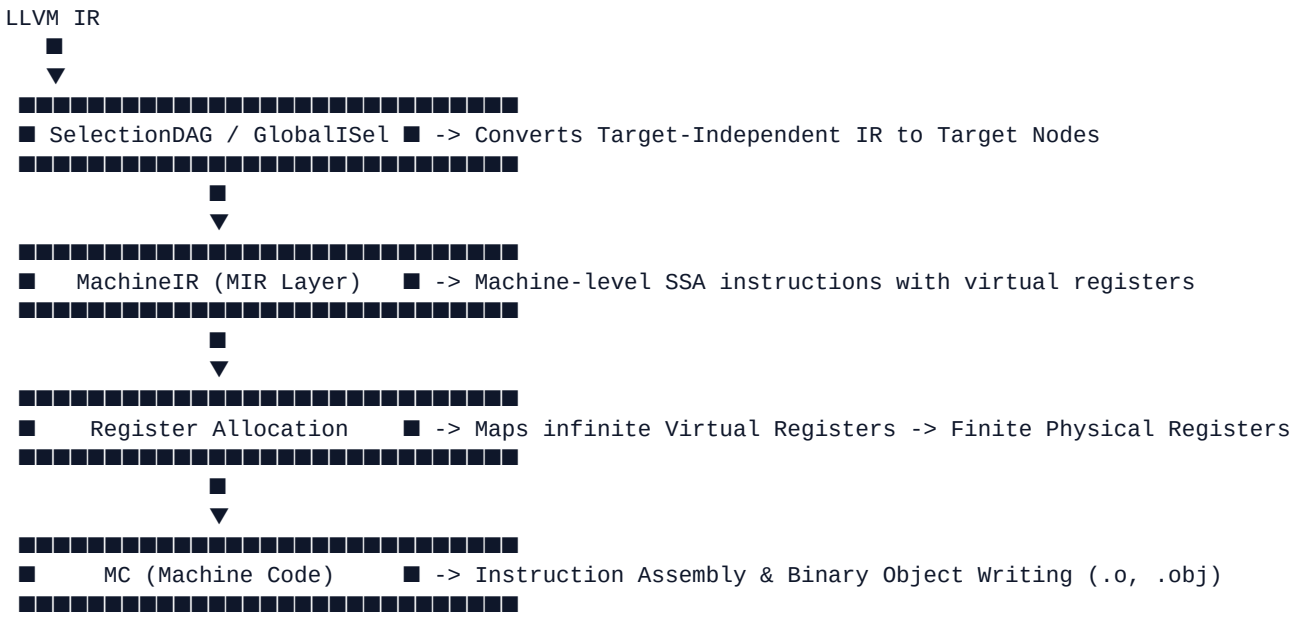
Chapter 13: LLVM Backend Architecture: SelectionDAG, GlobalISel & MachineIR

> **Core Literature Grounding**: *LLVM Code Generation: A Deep Dive into Compiler Backend Development* by Quentin Colombet

> **Compiler Module Focus**: `agam_codegen`

13.1 Overview of the LLVM Target Backend Architecture

Quentin Colombet's definitive work details how LLVM translates target-independent LLVM IR into physical, hardware-specific machine instructions:



13.2 SelectionDAG vs. GlobalISel

1. **SelectionDAG (Legacy Pipeline)**:

- Constructs a Directed Acyclic Graph (DAG) for each Basic Block.

- Performs **Type Legalization** (splits unsupported types like `i128` into `i64` pairs) and **DAG Combine** optimizations.

- Translates DAG nodes into target instructions using pattern matching defined in TableGen files (`.td`).

2. **GlobalISel (Global Instruction Selection Framework)**:

- Designed and architected by Quentin Colombet.

- Operates globally across whole functions rather than basic blocks.

- Operates directly on **MachineIR (MIR)** using four fast sequential passes: `IRTranslator` $\rightarrow$ `Legalizer` $\rightarrow$ `RegisterBankSelect` $\rightarrow$ `InstructionSelect`.

13.3 TableGen (`.td`) Target Descriptions

LLVM target instruction sets (x86_64, AArch64, RISC-V) are declared using the **TableGen** domain-specific language (`.td` files).

TableGen defines:

- **Register Classes**: `GR64` (`rax`, `rbx`, `rcx`), `FR64` (`xmm0`–`xmm15`).

- **Instruction Definitions**: Opcode encodings, register constraints, side effects.

- **Pattern Matching Rules**: Mapping IR operations directly to hardware opcodes.

```
// Example TableGen pattern matching 64-bit addition on x86
def ADD64rr : I<0x01, MRMDestReg, (outs GR64:$dst), (ins GR64:$src1, GR64:$src2),
                "add{q}\t{$src2, $dst|$dst, $src2}",
                [(set GR64:$dst, (add GR64:$src1, GR64:$src2))]>;
```

Chapter 14: Register Allocation Algorithms & Machine Code (MC) Layer

> **Core Literature Grounding**: *LLVM Code Generation: A Deep Dive into Compiler Backend Development* by Quentin Colombet

> **Compiler Module Focus**: `agam_codegen`

14.1 The Register Allocation Problem

Target CPUs possess a strictly finite number of physical registers (e.g., 16 general-purpose registers on x86_64, 31 on AArch64). However, MachineIR (MIR) instructions operate on an infinite set of **Virtual Registers** (`%vreg0`, `%vreg1`).

Register Allocation maps virtual registers to physical hardware registers while minimizing memory spill operations.

14.2 Register Allocation Algorithms

1. Graph Coloring Register Allocation (Chaitin-Briggs)

1. **Liveness Analysis**: Computes live ranges for all virtual registers.

2. **Interference Graph Construction**: Constructs a graph $G=(V, E)$ where vertices V represent virtual registers and edges E represent overlapping live ranges.

3. **Graph Coloring (K-Coloring)**: Colors the graph using K physical registers.

4. **Spilling**: If the graph chromatic number exceeds K, virtual registers with low use intensity are spilled to stack memory (`mov [rsp+16], rax`).

2. Greedy Register Allocator (LLVM Production Allocator)

LLVM's production allocator processes live ranges in priority order based on execution frequency, splitting live ranges across basic block boundaries to minimize spill code overhead.

14.3 The MC (Machine Code) Layer

The **MC Layer** is LLVM's lowest level component. It converts physical `MCInst` instructions into binary object files (`.o`, `.obj` in ELF, COFF, or Mach-O format) and resolves symbol relocations (`R_X86_64_PC32`).

Chapter 15: End-to-End Agam Compiler Pipeline Walkthrough

> **System Scope**: Full Agam Compiler Lifecycle & Driver Architecture

> **Compiler Module Focus**: `agam_driver`, `agam_pkg`

15.1 Complete Source-to-Binary Execution Flow

The `agamc` CLI orchestrates the full compilation lifecycle:

15.2 Driver Coordination & Command CLI (`agam_driver`)

The CLI entrypoint (`agamc`) handles key developer workflows:

| CLI Command | Action Executed | Primary Crate Targets |

| :--- | :--- | :--- |

| `agamc build` | Complete compilation to native binary executable | `agam_driver` $\rightarrow$ `agam_codegen` |

| `agamc run` | Build and execute target binary | `agam_driver` $\rightarrow$ `agam_runtime` |

| `agamc check` | Fast type checking and diagnostic verification | `agam_lexer` $\rightarrow$ `agam_sema` |

| `agamc repl` | Interactive REPL buffer execution | `agam_driver` $\rightarrow$ `agam_jit` |

| `agamc dev` | Incremental warm-daemon execution loop | `agam_driver` $\rightarrow$ `DaemonSession` |

| `agamc exec` | Sandboxed headless agent execution | `agam_driver` $\rightarrow$ `agam_notebook` |

| `agamc doctor` | Host system LLVM and C toolchain verification | `agam_driver` $\rightarrow$ `agam_runtime` |

Chapter 16: Advanced Language Features: Native Tensors & Algebraic Effects

> **System Scope**: Agam First-Class Language Primitives

> **Compiler Module Focus**: `agam_ast`, `agam_sema`, `agam_mir`

16.1 Native Tensor Operations

In Agam, multi-dimensional numerical tensors are first-class compiler primitives rather than external C++ library bindings.

```
let A: Tensor[Float, 2x3] = Tensor.from_array([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0]
]);

let B: Tensor[Float, 3x2] = Tensor.ones([3, 2]);

// Native matrix multiplication compiled directly to SIMD / BLAS instructions
let C = A * B;
```

Compiler Lowering for Tensors

1. **Type Checker (`agam_sema`)**: Verifies tensor dimension compatibility at compile time ($\text{InnerDim}(A) == \text{OuterDim}(B)$).

2. **MIR Generation (`agam_mir`)**: Emits SIMD loop constructs or BLAS external call nodes (`agam_runtime_matmul`).

16.2 Algebraic Effect Handlers (`perform`, `handle`, `effect`)

Agam implements algebraic effect handlers (`Phase 20`), allowing control flow and side-effects to be handled modularly without callback structures:

```
effect Database {
    fn query(sql: String) -> String;
}
```

```
fn fetch_user_profile(id: Int) -> String {
    perform Database.query("SELECT * FROM users WHERE id = " + id.to_string());
    return "User Profile";
}

fn main() {
    handle fetch_user_profile(42) {
        Database.query(sql) => {
            println("Intercepted SQL: " + sql);
            resume("Mocked Result"); // Resumes execution back to caller
        }
    }
}
```

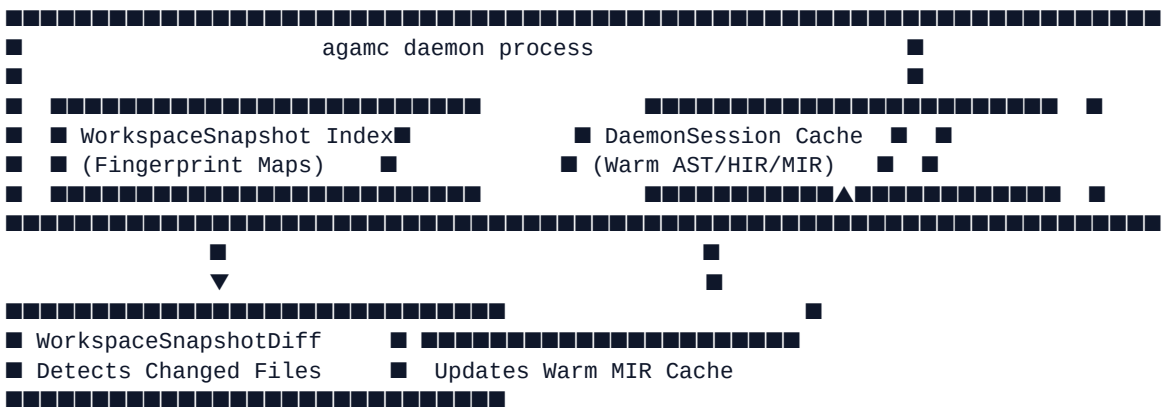
Chapter 17: Incremental Compilation Daemon & Sandboxed Execution

> **System Scope**: Tooling Infrastructure & Security Hardening

> **Compiler Module Focus**: `agam_driver`, `agam_pkg`, `agam_runtime`

17.1 Incremental Background Daemon (`Phase 15F`)

To deliver sub-millisecond compile loops during development, `agamc` runs a background daemon process (`DaemonSession`):



- `WorkspaceSnapshot`: Fingerprints source file contents to detect modifications instantly.

- `DaemonSession`: Holds pre-parsed ASTs, HIR, and serialized MIR artifacts in warm memory, eliminating redundant parsing of unchanged workspace modules.

- **IPC TCP Loopback (`127.0.0.1:0`)**: Standard binary CLI commands query the background daemon over localhost TCP sockets.

17.2 Sandboxed Execution Hardening (`Phase 21`)

When executing untrusted user code or running headless agent tool calls (`agamc exec`), the Agam runtime enforces strict operating system-level process sandboxing:

- **Windows Platform**: Enforces Windows `JobObject` limits restricting maximum memory usage, CPU rate limits, and child process creation.

- **Linux Platform**: Invokes `prctl` (`PR_SET_NO_NEW_PRIVS`) and `setrlimit` syscalls to restrict RAM allocation, file descriptor counts, and execution timeouts.

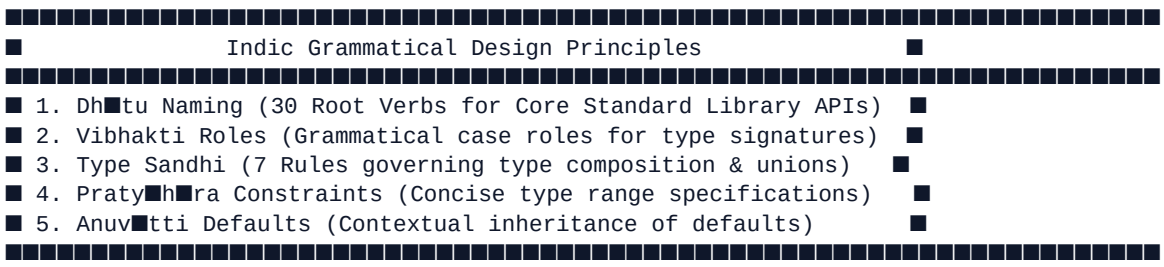
Chapter 18: Indic Grammatical Design Principles (P■■ini & Tolk■ppiyam)

> **System Scope**: Theoretical Design Philosophy (`Phase F6`)

> **Compiler Module Focus**: `docs/specification/design-principles.md`

18.1 Grammatical Principles in Programming Language Design

Agam formalizes seven core language design principles derived from **P■■ini's A■■■dhy■y■** (Sanskrit) and the **Tolk■ppiyam** (Tamil) — the world's oldest formal grammar systems:



18.2 Dh■tu Root Verbs & Vibhakti Roles

1. Dh■tu Naming Conventions

The standard library API surface is systematically derived from 30 canonical root verbs (*Dh■tus*), establishing semantic consistency across all modules:

- k■ (Do/Make) $\rightarrow$ Construct, initialize

- `grah` (Take/Receive) $\rightarrow$ Fetch, parse, extract

- d■ (Give/Emit) $\rightarrow$ Return, yield, emit

2. Vibhakti Roles (Grammatical Cases)

Type parameters and function arguments follow grammatical case roles:

- **Agent (Kart■)**: Invoking context

- **Patient/Object (Karman)**: Data target operated upon

- **Instrument (Kara■a)**: Options or configuration parameters

---

18.3 Type Sandhi Rules

Type Sandhi establishes formal rules for type composition, union merging, and automatic type coercions:

1. **Vowel Sandhi (Homogeneous Join)**: Merging identical primitive types ($T \cup T \implies T$).

2. **Consonant Sandhi (Subtype Coercion)**: Coercing bounded subtypes to common supertypes.

3. **Visarga Sandhi (Option Transformation)**: Merging optional types ($T \cup \text{Nil} \implies \text{Option}[T]$).

Chapter 19: Getting Started & Basics of Agam

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Software Engineers learning to write code in Agam (Basic Level)

19.1 Introduction to Agam Programming

Agam is a next-generation compiled programming language designed to combine Python-level readability, Rust-level memory safety, and C/LLVM native execution speed.

Key language characteristics:

- **Static Typing with Local Type Inference**: Strongly typed at compile time without verbose annotations.

- **Native Tensor Operations**: Multi-dimensional numerical arrays are first-class language constructs.

- **Algebraic Effect Handlers**: Structured side-effect management replacing callbacks and complex error hierarchies.

- **Zero-Overhead Memory Safety**: Automatic memory management without a global stop-the-world garbage collector.

19.2 "Hello, World!" in Agam

Create a file named `hello.agam`:

```
fn main() {
    println("Hello, Agam World!");
}
```

Compiling and Running

Use the `agamc` CLI tool:

```
# Build a native standalone binary
agamc build hello.agam

# Run directly
agamc run hello.agam
```

19.3 Variables, Mutability & Constants

In Agam, variables are declared using `let` and are **immutable by default**. To make a variable mutable, append `mut`:

```
fn main() {
    // Immutable variable binding
    let name: String = "Agam";
    let version = 1; // Type inferred as Int

    // Mutable variable binding
    let mut score: Int = 100;
    score = score + 50;

    // Constants (evaluated at compile time)
    const MAX_CONNECTIONS: Int = 1024;

    println("Language: " + name);
    println("Score: " + score.to_string());
}
```

19.4 Primitive Data Types

Agam supports primitive types:

| Type | Description | Example |

| :--- | :--- | :--- |

| `Int` | 64-bit signed integer | `42`, `-100` |

| `Float` | 64-bit IEEE 754 floating point | `3.14159`, `-0.5` |

| `Bool` | Boolean truth value | `true`, `false` |

| `Char` | 32-bit Unicode scalar character | `'A'`, `'α'` |

| `String` | UTF-8 encoded text string | `"Agam Language"` |

| `Nil` | Unit/empty value | `()` |

19.5 Functions & Signatures

Functions are declared with `fn`, followed by parameter names, type annotations, and an optional return type (`-> Type`):

```
// Function taking arguments and returning a Float
fn calculate_bmi(weight_kg: Float, height_m: Float) -> Float {
    let bmi = weight_kg / (height_m * height_m);
    return bmi;
}

// Single-expression implicit return syntax
fn add(a: Int, b: Int) -> Int => a + b;

fn main() {
    let result = calculate_bmi(70.0, 1.75);
    println("BMI Result: " + result.to_string());
}
```

Chapter 20: Control Flow, Structs & Collections

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Software Engineers learning Agam (Intermediate Level)

20.1 Control Flow: Conditionals & Loops

1. `if` Expression

In Agam, `if` is an expression that returns a value:

```
fn main() {
    let score = 85;

    // Conditionals return values directly
    let status = if score >= 50 {
        "Passed"
    } else {
        "Failed"
    };

    println("Status: " + status);
}
```

2. `while` and `for` Loops

```
fn main() {
    // Standard while loop
    let mut count = 0;
    while count < 5 {
        println("Count: " + count.to_string());
        count = count + 1;
    }

    // Range-based for loop
    for i in 0..5 {
        println("Iteration: " + i.to_string());
    }
}
```

20.2 Composite Structures (`struct`)

Structs group related data fields into custom types:

```
struct User {
    username: String,
    email: String,
    age: Int,
    is_active: Bool,
}

// Associated methods implementation block
impl User {
    fn new(name: String, email: String, age: Int) -> User {
        return User {
            username: name,
            email: email,
            age: age,
            is_active: true,
        };
    }

    fn deactivate(self) -> User {
        return User {
            username: self.username,
            email: self.email,
            age: self.age,
            is_active: false,
        };
    }
}

fn main() {
    let user1 = User.new("Alice", "alice@example.com", 28);
    println("User: " + user1.username);
}
```

20.3 Arrays & Tuples

```
fn main() {
    // Fixed-size homogeneous Array
    let numbers: Array[Int] = [10, 20, 30, 40, 50];
    println("First element: " + numbers[0].to_string());

    // Heterogeneous Tuple
    let pair: (String, Int) = ("Score", 99);
    println("Label: " + pair.0 + ", Value: " + pair.1.to_string());
}
```

Chapter 21: Tagged Union Enums, Pattern Matching & Error Handling

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Software Engineers learning Agam (Intermediate-to-Advanced)

21.1 Tagged Union Enums

Enums in Agam can carry payload data inside variant constructors:

```
enum Command {
    Quit,
    Move { x: Int, y: Int },
    Write(String),
    ChangeColor(Int, Int, Int),
}
```

21.2 Pattern Matching (`match`)

Pattern matching is exhaustive; every possible enum variant must be handled:

```
fn process_command(cmd: Command) {
    match cmd {
        Command.Quit => println("Quitting program..."),
        Command.Move { x, y } => {
            println("Moving to X: " + x.to_string() + ", Y: " + y.to_string());
        }
        Command.Write(text) => println("Writing text: " + text),
        Command.ChangeColor(r, g, b) => println("Color changed"),
    }
}
```

21.3 Robust Error Handling with `Option` and `Result`

Agam avoids `null` pointer exceptions by using explicit `Option[T]` and `Result[T, E]` types:

```
enum Option[T] {
```

```
    Some(T),
    None,
}

enum Result[T, E] {
    Ok(T),
    Err(E),
}

fn divide(numerator: Float, denominator: Float) -> Result[Float, String] {
    if denominator == 0.0 {
        return Result.Err("Division by zero error");
    }
    return Result.Ok(numerator / denominator);
}

fn main() {
    match divide(10.0, 2.0) {
        Result.Ok(val) => println("Division result: " + val.to_string()),
        Result.Err(err) => println("Error occurred: " + err),
    }
}
```

Chapter 22: First-Class Tensors & Numerical AI Operations

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: AI / ML Engineers and Numerical Computing Developers (Advanced Level)

22.1 First-Class Tensor Primitives

In Agam, multi-dimensional numerical arrays (`Tensor`) are native primitives integrated into the syntax and backend compiler code generator (`agam_codegen`).

```
fn main() {
    // 2D Matrix Creation
    let A: Tensor[Float, 2x3] = Tensor.from_array([
        [1.0, 2.0, 3.0],
        [4.0, 5.0, 6.0]
    ]);

    let B: Tensor[Float, 3x2] = Tensor.from_array([
        [7.0, 8.0],
        [9.0, 1.0],
        [2.0, 3.0]
    ]);

    // Matrix Multiplication compiled directly to SIMD / BLAS kernels
    let C: Tensor[Float, 2x2] = A * B;

    println("Result Matrix shape: " + C.shape().to_string());
}
```

22.2 Tensor Broadcasting & Arithmetic

Agam supports element-wise mathematical operations with automatic shape broadcasting:

```
fn main() {
    let X = Tensor.ones([4, 4]); // 4x4 matrix of 1.0s
    let bias = Tensor.from_array([0.5, 1.0, 1.5, 2.0]); // 1x4 vector

    // Automatic broadcasting across rows
    let Y = X + bias;
    let Z = Tensor.relu(Y); // Native Rectified Linear Unit activation
}
```

22.3 Neural Network Layer Construction

```
struct LinearLayer {
    weights: Tensor[Float],
    bias: Tensor[Float],
}

impl LinearLayer {
    fn new(in_features: Int, out_features: Int) -> LinearLayer {
        return LinearLayer {
            weights: Tensor.random([in_features, out_features]),
            bias: Tensor.zeros([out_features]),
        };
    }

    fn forward(self, input: Tensor[Float]) -> Tensor[Float] {
        return (input * self.weights) + self.bias;
    }
}
```

Chapter 23: Algebraic Effect Handlers in Depth

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Advanced Software Engineers & Systems Architects

---

23.1 What Are Algebraic Effects?

Algebraic Effects separate side-effect requests from their concrete implementations. Rather than hardcoding I/O calls, database accesses, or asynchronous polling inside business logic, functions invoke `perform Effect()`. Parent callers intercept effects using `handle` blocks.

Benefits:

- **Testability**: Intercept network calls during testing with zero code changes.

- **Resumable Control Flow**: Unlike exceptions which abort execution, effect handlers can `resume(value)` back to the exact call site.

23.2 Defining & Performing Effects

```
// 1. Declare Effect Signatures
effect Logger {
    fn log(msg: String) -> Nil;
}

effect Fetcher {
    fn get_url(url: String) -> String;
}

// 2. Function performs effects without knowing who handles them
fn ProcessData(url: String) -> String {
    perform Logger.log("Initiating fetch for: " + url);
    let raw_data = perform Fetcher.get_url(url);
    perform Logger.log("Fetch complete. Bytes received: " + raw_data.length().to_string());
    return raw_data;
}
```

23.3 Intercepting Effects with `handle` and `resume`

```
fn main() {
    // 3. Handle effects at top-level caller
    handle ProcessData("https://api.example.com/data") {
        Logger.log(msg) => {
            println("[LOG INTERCEPTED]: " + msg);
            resume(); // Continue execution after log call
        },
        Fetcher.get_url(url) => {
            println("[MOCK FETCHER]: Mocking request to " + url);
            resume("{ \"status\": \"success\", \"data\": 42 }"); // Pass return value to perform
        }
    }
}
```

23.4 Async Effect Handlers

Algebraic effects naturally model asynchronous non-blocking I/O without requiring `async`/`await` keyword clutter throughout the codebase. The runtime handler suspends computation until I/O events complete, then resumes execution transparently.

Chapter 24: Modules, Package Management (`agam.toml`) & FFI

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Systems Engineers and Application Developers

---

24.1 Package Manifests (`agam.toml`)

Agam projects use `agam.toml` for package declaration and dependency management:

```
[project]
name = "my_ai_app"
version = "0.1.0"
authors = ["Developer <dev@example.com>"]
edition = "2026"

[dependencies]
std = "1.0"
math_utils = { path = "../math_utils" }
network_pkg = { git = "https://github.com/example/network_pkg.git", tag = "v1.2.0" }

[toolchain]
llvm_version = "18.1"
```

24.2 Modules & Code Importing

Split code across multiple files:

```
// File: src/math.agam
pub fn add_vectors(a: Array[Float], b: Array[Float]) -> Array[Float] {
    // ...
}

// File: src/main.agam
import src.math as math;

fn main() {
    let result = math.add_vectors([1.0], [2.0]);
}
```

24.3 Foreign Function Interface (FFI) Interop

Agam can interface directly with external C libraries or Python frameworks (`agam_ffi`):

Calling C Native Libraries

```
extern "C" {
    fn puts(str: *const Char) -> Int;
    fn malloc(size: Int) -> *mut Nil;
    fn free(ptr: *mut Nil);
}

fn main() {
    unsafe {
        puts("Direct C string call via FFI");
    }
}
```

Chapter 25: Metaprogramming, REPL, Notebooks & Tooling

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Advanced Developers & AI Practitioners

---

25.1 Interactive JIT REPL (`agamc repl`)

Launch the interactive REPL for rapid evaluation:

```
$ agamc repl
Agam v0.1.0 Interactive REPL
>>> let x = 42
x: Int = 42
>>> x * 2
84
```

25.2 Headless Agent Execution (`agamc exec`)

Execute Agam scripts in headless JSON-stream mode with strict resource limits for AI agent workflows:

```
agamc exec --json '{"source": "println(40 + 2)", "memory_limit_mb": 512}'
```

25.3 Formatter & Linter

```
# Format source code
agamc fmt src/main.agam

# Run compiler static linter
agamc lint src/main.agam
```

Chapter 25b: Real-World Agam Code Cookbook

> **Part VI: The Agam Language Programming Guide**

> **Target Audience**: Software Engineers building production applications in Agam

---

Recipe 1: Production Web API Handler using Algebraic Effects

This recipe builds an HTTP API handler where database queries and logging side-effects are cleanly decoupled via algebraic effect handlers:

```
// 1. Declare Effect Interfaces
effect Database {
    fn find_user_by_id(id: Int) -> Option[String];
}

effect Logger {
    fn info(msg: String) -> Nil;
}

// 2. Pure Business Logic Function
fn handle_user_request(user_id: Int) -> String {
    perform Logger.info("Received API request for user ID: " + user_id.to_string());

    match perform Database.find_user_by_id(user_id) {
        Option.Some(user_json) => {
            perform Logger.info("User successfully found.");
            return "{ \"status\": 200, \"data\": " + user_json + " }";
        },
        Option.None => {
            perform Logger.info("User ID not found in database.");
            return "{ \"status\": 404, \"error\": \"User Not Found\" }";
        }
    }
}

// 3. Application Entrypoint with Concrete Handlers
fn main() {
    println("--- Test 1: Existing User ---");
    handle handle_user_request(42) {
        Logger.info(msg) => {
            println("[LOG]: " + msg);
            resume();
        },
        Database.find_user_by_id(id) => {
            if id == 42 {
                resume(Option.Some("{ \"name\": \"Alice\", \"role\": \"Admin\" }"));
```

```
            } else {
                resume(Option.None);
            }
        }
    }
}
```

Recipe 2: Machine Learning Tensor Training Pipeline

This recipe constructs a 2-layer neural network forward pass using Agam's native tensors:

```
struct MultiLayerPerceptron {
    w1: Tensor[Float],
    b1: Tensor[Float],
    w2: Tensor[Float],
    b2: Tensor[Float],
}

impl MultiLayerPerceptron {
    fn new(in_dim: Int, hidden_dim: Int, out_dim: Int) -> MultiLayerPerceptron {
        return MultiLayerPerceptron {
            w1: Tensor.random([in_dim, hidden_dim]),
            b1: Tensor.zeros([hidden_dim]),
            w2: Tensor.random([hidden_dim, out_dim]),
            b2: Tensor.zeros([out_dim]),
        };
    }

    fn forward(self, x: Tensor[Float]) -> Tensor[Float] {
        // Layer 1: Linear + ReLU
        let h1 = Tensor.relu((x * self.w1) + self.b1);
        // Layer 2: Linear Output
        let out = (h1 * self.w2) + self.b2;
        return out;
    }
}

fn main() {
    let mlp = MultiLayerPerceptron.new(784, 128, 10);
    let sample_batch = Tensor.ones([32, 784]); // Batch size 32, 784 features

    let predictions = mlp.forward(sample_batch);
    println("Output Batch Tensor Shape: " + predictions.shape().to_string());
}
```

Chapter 26: Diagnostic Engineering, Spans & Error Recovery

> **Part VII: Advanced Tooling, Testing & Ecosystem Engineering**

> **Compiler Module Focus**: `agam_errors`

26.1 Diagnostic Architecture in Production Compilers

A compiler's diagnostic engine is often a developer's primary interface with the language. Cryptic or misaligned error reports slow development down significantly.

In the Agam Compiler, `agam_errors` provides a unified diagnostic infrastructure that:

- Captures exact source byte spans (`Span`, `SourceId`).

- Supports multi-line code snippet extraction and underline highlighting.

- Renders rich terminal output using ANSI colors and human-readable suggestions.

26.2 Diagnostic Data Models

```
#[derive(Debug, Clone)]
pub struct Diagnostic {
    pub level: DiagnosticLevel, // Error, Warning, Note, Help
    pub code: Option<String>,   // e.g., "E0308"
    pub message: String,
    pub primary_label: Label,
    pub secondary_labels: Vec<Label>,
    pub suggestions: Vec<Suggestion>,
}

#[derive(Debug, Clone)]
pub struct Label {
    pub span: Span,
    pub message: String,
}

#[derive(Debug, Clone)]
pub struct Suggestion {
    pub span: Span,
    pub replacement: String,
    pub message: String,
}
```

26.3 Diagnostic Rendering Example

When `agam_sema` detects a type mismatch, `agam_errors` renders a detailed report:

```
error[E0308]: mismatched types
 --> src/main.agam:12:21
   |
12 |     let count: Int = "forty-two";
   |                ---   ^^^^^^^^^^^^ expected `Int`, found `String`
   |                |
   |                expected due to this type annotation
   |
help: to convert a String to an Int, use `String.parse_int()`
   |
12 |     let count: Int = "forty-two".parse_int();
   |                                  ++++++++++++
```

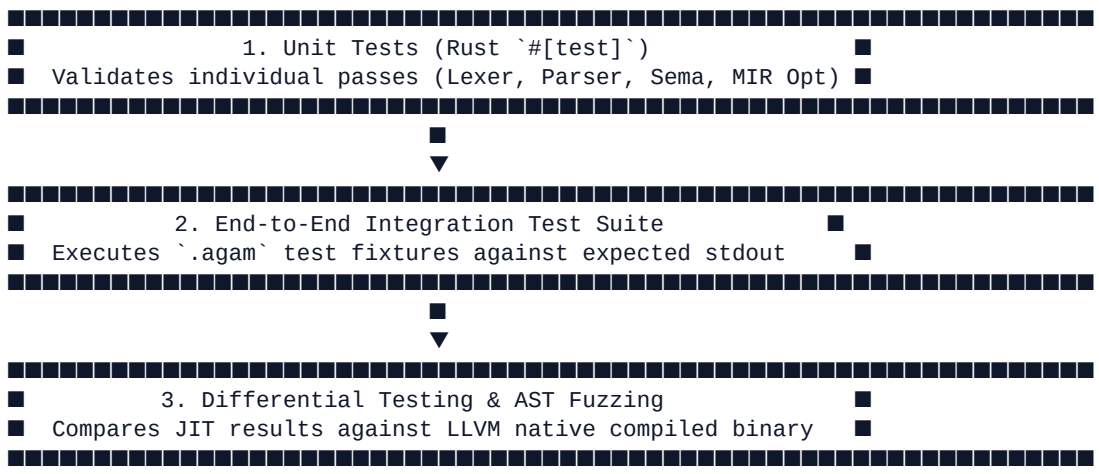
Chapter 27: Testing Methodologies, Fuzzing & Differential Verification

> **Part VII: Advanced Tooling, Testing & Ecosystem Engineering**

> **Compiler Module Focus**: `agam_test`

27.1 Multi-Tier Compiler Testing Framework

Compiler bugs can manifest as incorrect diagnostic reporting, silent code miscompilation, or unexpected crashes during code generation. `agam_test` enforces a multi-tier verification strategy:



27.2 Integration Test Harness (`agam_test`)

Integration tests use inline test annotations inside `.agam` files:

```
// RUN: agamc run %s | FileCheck %s
// CHECK: Calculated Result: 150

fn main() {
    let a = 100;
    let b = 50;
    println("Calculated Result: " + (a + b).to_string());
}
```

The test runner compiles each fixture, executes the generated binary, and compares `stdout` against `CHECK` directives.

27.3 Differential Verification

`agam_test` verifies correctness across different execution backends:

$$\text{Evaluate(Source, Backend::JIT)} \stackrel{?}{=} \text{Evaluate(Source, Backend::LLVM\_Native)}$$

If the Cranelift JIT engine produces a result that differs from the native LLVM machine executable, a differential test failure is flagged.

Chapter 28: Language Server Protocol (LSP) Architecture

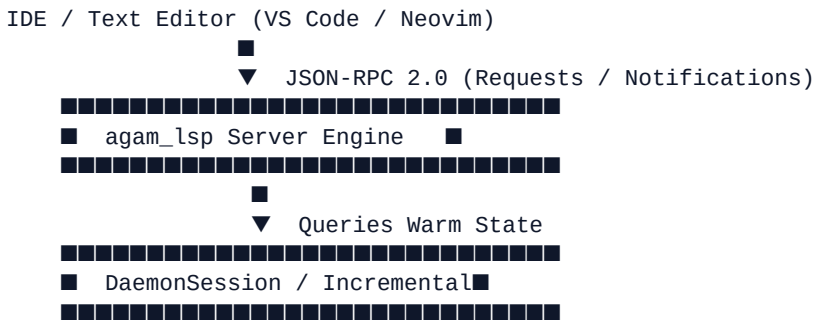
> **Part VII: Advanced Tooling, Testing & Ecosystem Engineering**

> **Compiler Module Focus**: `agam_lsp`

28.1 Overview of the Language Server Protocol

The **Language Server Protocol (LSP)** standardizes communication between code editors (VS Code, Neovim, Visual Studio, IntelliJ) and programming language compilers.

`agam_lsp` implements the LSP JSON-RPC server specification over stdin/stdout or TCP loopback, allowing IDEs to query compiler state in real time as developers edit files.



28.2 Key LSP Features Implemented in `agam_lsp`

1. `textDocument/publishDiagnostics`: Pushes real-time type errors, syntax warnings, and unhandled effect diagnostics to the editor canvas upon every keypress.

2. `textDocument/hover`: Provides hover tooltips displaying function signatures, variable inferenced types, and docstrings.

3. `textDocument/definition`: Navigates from symbol references directly to their source definition locations (`Span`).

4. `textDocument/completion`: Offers contextual autocomplete suggestions for struct fields, module functions, and keywords.

Chapter 29: Source Code Formatting Engine Architecture (`agam_fmt`)

> **Part VII: Advanced Tooling, Testing & Ecosystem Engineering**

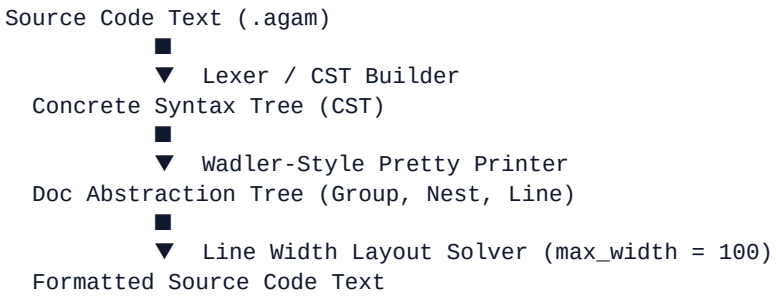
> **Compiler Module Focus**: `agam_fmt`

29.1 The Role of Code Formatters

Code formatters enforce a unified code style across codebases, eliminating style debates in pull requests and improving code readability.

Unlike compilers which discard white space and comments during AST parsing, a code formatter must operate on **Concrete Syntax Trees (CST)** or token streams that preserve comments, blank lines, and source layout.

29.2 Formatter Engine Pipeline



Formatter Algorithm Rules:

1. **Indentation**: Standard 4-space indent per block level.

2. **Line Breaking**: Wrap function arguments or struct fields across multiple indented lines when line length exceeds 100 characters.

3. **Comment Preservation**: Re-attach floating or inline comments (`//`, `/ /`) to their nearest sibling syntax nodes.

Chapter 30: Cross-Compilation, Target Triplets & Target Packs

> **Part VII: Advanced Tooling, Testing & Ecosystem Engineering**

> **Compiler Module Focus**: `agam_pkg`, `agam_codegen`

30.1 Target Triplets & Cross-Compilation

Agam supports compiling native binaries for cross-platform architectures. Targets are identified using **LLVM Target Triplets**:

```
x86_64-pc-windows-msvc      (Windows x64 Native)
x86_64-unknown-linux-gnu    (Linux x64 Native)
aarch64-linux-android       (Android ARM64 Target)
```

30.2 Target Packs & SDK Staging (`agamc package sdk`)

`agam_pkg` manages modular **Target Packs** (`Phase 15H`) containing sysroots, pre-compiled runtime static libraries (`libagam_runtime.a`), and LLVM target description files:

```
# Building an Android ARM64 target binary from a Windows host machine
agamc build --target aarch64-linux-android main.agam
```

The driver configures LLVM target machine triples, configures cross-linker parameters, and packages output binaries into release-ready `.apk` or `.agpkg` archives.

Chapter 31: Compiler Profiling & Performance Measurement

> **Part VII: Advanced Tooling, Testing & Ecosystem Engineering**

> **Compiler Module Focus**: `agam_profile`

31.1 Compiler Performance Methodology

Claims about compiler optimization performance gains must be verified through empirical measurement, not intuition.

`agam_profile` provides automated benchmarking harnesses and profiling instrumentation to track two distinct metrics:

1. **Compilation Speed (Throughput)**: Time taken to parse, type check, optimize, and generate code ($\text{Lines of Code / Second}$).

2. **Generated Binary Speed (Execution Latency)**: Runtime performance of optimized target binary code compared against `clang++ -O3` C++ implementations.

31.2 Flamegraph & Phase Timings

When running `agamc build --profile`, `agam_profile` records duration metrics across compiler phases:

```
=====================================================
            AGAM COMPILER PROFILE SUMMARY
=====================================================
  Phase 1: Lexer & Parsing          :   1.2 ms  ( 4%)
  Phase 2: Semantic Analysis        :   3.1 ms  (10%)
  Phase 3: HIR & MIR Lowering       :   2.8 ms  ( 9%)
  Phase 4: MIR Optimizations        :   4.5 ms  (15%)
  Phase 5: LLVM IR & Codegen        :  18.4 ms  (62%)
-----------------------------------------------------
  TOTAL COMPILATION TIME            :  30.0 ms
=====================================================
```

Appendix A: Comprehensive Agam Crate Reference

> **Physical Location**: `crates/{core,middle,backends,runtime,tooling,experiments}`

Workspace Crate Breakdown

1. Core Crates (`crates/core`)

- `agam_errors`: Centralized diagnostic reporting, `Span`, `SourceId`, color highlighting.

- `agam_lexer`: Lexical scanner, token stream generation, UTF-8 position tracking.

- `agam_parser`: Pratt expression parser and statement parser.

- `agam_ast`: Abstract Syntax Tree node definitions and visitor traits.

2. Middle-End Crates (`crates/middle`)

- `agam_sema`: Symbol resolution, nested scope graph, type checker, effect checker.

- `agam_hir`: High-Level IR, pattern match desugaring.

- `agam_mir`: Medium-Level IR, Basic Blocks, CFG, SSA form, `agam_mir::opt` optimization passes.

3. Backend Crates (`crates/backends`)

- `agam_codegen`: LLVM IR lowering, C99 portable fallback code emitter.

- `agam_jit`: In-process Cranelift & LLVM ORC JIT execution engine.

4. Runtime Crates (`crates/runtime`)

- `agam_runtime`: C ABI bindings, memory allocator primitives, host detection.

- `agam_std`: Standard library runtime definitions.

5. Tooling Crates (`crates/tooling`)

- `agam_driver`: Main `agamc` CLI executable driver and `DaemonSession`.

- `agam_pkg`: `agam.toml` manifest handling, lockfile resolver (`agam.lock`).

- `agam_lsp`: Language Server Protocol implementation.
- `agam_fmt`: Source code formatter.

Appendix B: Annotated Bibliography & Reading List

Landmark Compiler Literature References

1. **Kernighan, Brian W., and Dennis M. Ritchie.** *The C Programming Language*. 2nd ed., Prentice Hall, 1988.

- *Essential Reading*: Explains stack layout, pointers, structures, alignment, and low-level execution models.

2. **Nystrom, Robert.** *Crafting Interpreters*. Genever Benning, 2021.

- *Essential Reading*: Best modern guide for readable lexer, Pratt parser, AST, and virtual machine design.

3. **Parr, Terence.** *Language Implementation Patterns: Create Your Own Domain-Specific and General-Purpose Languages*. Pragmatic Bookshelf, 2009.

- *Essential Reading*: Pattern catalog for AST structures, symbol tables, lexical scopes, and type checkers.

4. **Cooper, Keith D., and Linda Torczon.** *Engineering a Compiler*. 3rd ed., Morgan Kaufmann, 2022.

- *Essential Reading*: Definitive modern reference for IRs, Control Flow Graphs, SSA form, and middle-end optimization passes.

5. **Appel, Andrew W.** *Modern Compiler Implementation in C*. Cambridge University Press, 1998.

- *Essential Reading*: Practical guide for lowering functional semantics into imperative IRs and target assembly.

6. **Colombet, Quentin.** *LLVM Code Generation: A Deep Dive into Compiler Backend Development*. Packt Publishing, 2024.

- *Essential Reading*: Definitive guide on GlobalISel, SelectionDAG, MachineIR (MIR), TableGen (`.td`), and target backend codegen.

7. **Nacke, Kai, and Amy Kwan.** *LLVM Techniques, Tips, and Best Practices*. Packt Publishing, 2021.

- *Essential Reading*: Practical API manual for LLVM C++ builder patterns, modern PassManager, and ORC JIT engines.

Appendix C: Glossary of Compiler & Indic Design Terms

---

1. Compiler Engineering Terms

- **Abstract Syntax Tree (AST)**: A tree representation of source code syntax that omits concrete formatting noise (commas, semicolons, parentheses) while preserving structural semantics.

- **Application Binary Interface (ABI)**: A low-level machine contract defining parameter passing, register usage, stack frame layout, and return value mechanics between compiled binary modules.

- **Basic Block**: A straight-line sequence of instructions with a single entry point (first instruction) and a single exit point (terminator instruction).

- **Control Flow Graph (CFG)**: A directed graph where basic blocks form nodes and jump instructions (`Branch`, `Goto`, `Return`) form edges.

- **Dead Code Elimination (DCE)**: An optimization pass that removes instructions or basic blocks whose values are never consumed during execution.

- **Dominance Frontier (DF)**: The set of basic blocks where a node's dominance stops, determining exact placement locations for SSA ϕ-nodes.

- **GlobalISel**: LLVM's modern global instruction selection framework developed by Quentin Colombet, operating directly on whole-function MachineIR.

- **Intermediate Representation (IR)**: Target-independent code representations (HIR, MIR, LLVM IR) used between frontend parsing and backend codegen.

- **Pratt Parsing**: Top-Down Operator Precedence parsing associating binding powers with infix and prefix tokens to resolve operator precedence cleanly.

- **SelectionDAG**: LLVM's legacy basic-block DAG-based instruction selection engine.

- **Static Single Assignment (SSA)**: An IR property guaranteeing that every variable is defined exactly once, using ϕ-nodes to merge values at control flow join points.

---

2. Indic Grammatical Design Terms (P■■ini & Tolk■ppiyam)

- **A■■■dhy■y■**: P■■ini's 4th-century BCE Sanskrit grammar treatise consisting of ~4,000 formal generative rules, serving as the world's oldest formal grammar system.

- **Dh■tu (Root Verb)**: Canonical verbal roots (e.g., k■, `grah`, d■) used in Agam's standard library naming system for semantic consistency.

- **Praty■h■ra**: Concise shorthand notation for defining constrained sets or type sub-ranges.

- **Sandhi (Type Sandhi)**: Rules governing the composition, union, and transformation of types during type checking.

- **Tolk■ppiyam**: The oldest extant Tamil grammatical work, detailing phonology, morphology, syntax, and structural semantics.

- **Vibhakti (Grammatical Case)**: Case roles (Kart■/Agent, Karman/Object, Kara■a/Instrument) used to formalize function parameter roles.
